

Scorm (new): Syntax Analysis

Teacher: MEng. Tran Ngoc Bao Duy

Content

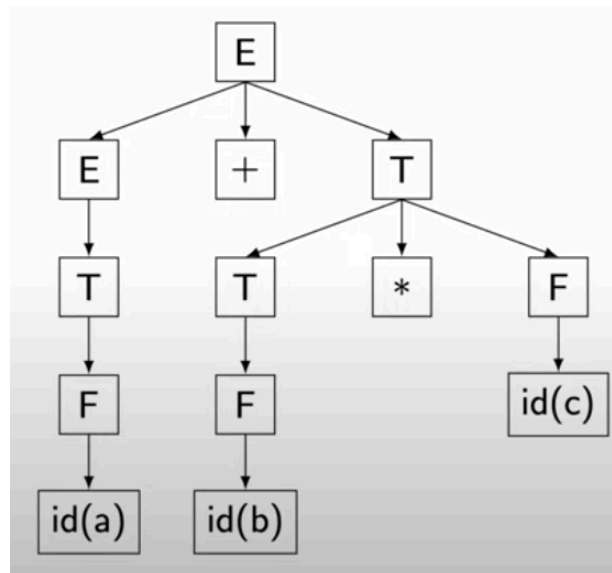
1. **Introduction to Syntax Analysis**
 - a. **Definition and Usage**
2. **Context-free Grammar (CFG)**
 - a. **The limit of RegEx**
 - b. **Symmetry in Programming**
 - c. **CFG: Definition**
 - d. **CFG: Example**
 - e. **BNF Vs. EBNF**
3. **Parser: How it works**
 - a. **How a parser uses CFG**
 - b. **Leftmost derivation: Definition**
 - c. **Leftmost derivation: Example**
4. **Parser generation from ANTLR4**
5. **Challenges (Ambiguous Grammar)**
6. **Common Patterns in Writing Parser Rules**
 - a. **A Non-empty list of x**
 - b. **A Null-able list of x**
 - c. **A Non-empty list of x, seperated by y**
 - d. **A Null-able list of x, seperated by y**
 - e. **An infix expression**

Introduction to Syntax Analysis

Definition and Usage

- **Syntax Analysis (phân tích cú pháp)** - hay còn được gọi là bộ **parser/parsing**, là bước thứ hai trong các bước dịch của compiler, tiếp nối của lexical analysis.

- Nhiệm vụ chính của parser là **kiểm tra** xem **trình tự** của các **tokens** tạo bởi lexer **có phù hợp với bất kỳ cấu trúc hợp lệ nào được nêu trong bộ quy tắc ngữ pháp của ngôn ngữ hay không**. **Nếu có một cấu trúc hợp lệ, parser sẽ sinh ra parse tree (cây cú pháp) để biểu diễn cho chuỗi token đó**. **Ngược lại, parser có nhiệm vụ thông báo lỗi cho người phát triển để tiến hành sửa code**.
- Vận dụng định nghĩa trên, khi gặp một bộ token được sinh ra từ lexer, parser sẽ tiến hành:
 - Kiểm tra xem bộ tokens có phù hợp với một trong các quy tắc ngữ pháp của ngôn ngữ hay không.
 - Nếu phù hợp, tiến hành sinh cây parse tree minh họa cho bộ token đó.
 - Nếu không phù hợp, phát hiện lỗi cú pháp và thông báo cho nhà phát triển.
- Ví dụ, với input là $a + b * c$, nếu được parser công nhận hợp lệ, parser sẽ sinh ra cây cú pháp sau:



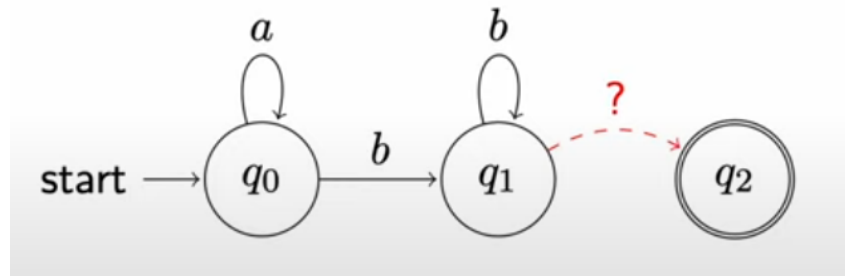
Context-free Grammar (CFG)

The limitation of Regular Expression

- Regular Expression (Regex)** rất mạnh cho vấn đề **khớp mẫu (pattern matching)**, nhưng lại yếu về các quy tắc đối xứng. **Một Regex không thể biểu diễn các vấn đề liên quan tới dữ liệu đối xứng như các dấu ngoặc nhọn/vuông/tròn lồng nhau; các vòng lặp; các loại lệnh điều kiện**.
- Vì vậy, Regex không đủ để biểu diễn các rules phức tạp của một ngôn ngữ lập trình, đòi hỏi có một phương thức khác cho phép đệ quy để thực hiện được những rules đối xứng.
- Xét ví dụ kinh điển về điểm yếu của Regex:

$$L = \{a^n b^n \mid n > 0\}$$

- Sử dụng automata để mô tả cho RegEx này như sau:



- a và b cùng có số mũ là n , automata trên không thể hiện được tính chất này.
- Ngoài ra, rõ ràng không thể xác định được cách đi từ q_1 statement tới end stament q_2 , bởi vì không xác định được số b là bao nhiêu. Automata này không thể hoạt động.

Symmetry in Programming

- Đối xứng (symmetry) trong lập trình tức là một phần phải “phản chiếu” hoặc liên kết với một phần khác để đảm bảo tính cân bằng và đúng đắn của ngôn ngữ.**
- Một vài ví dụ về đối xứng trong ngôn ngữ lập trình:
 - Nested Parentheses: `(( ))`
 - Các lệnh điều khiển: `if ... then ... else ; repeat ... until ...`
 - HTML Tags
- Regular Expression không thể mô tả được cấu trúc đối xứng.

Context-free grammar (CFG)

- Definition:** **Văn phạm phi ngữ cảnh (context-free grammar - CFG)** là một hệ thống chính thức được sử dụng để định nghĩa các ngôn ngữ lập trình, ngôn ngữ tự nhiên, và cấu trúc trong tính toán.
- Usage:** CFG sinh ra các chuỗi bằng các **áp dụng luật sinh (production rules)** - thứ sẽ thay thế những tín hiệu không kết thúc bằng chuỗi tín hiệu kết thúc.
 - Tín hiệu kết thúc** (terminal symbol): là các **token** được sinh ra từ lexer.
 - Tín hiệu không kết thúc** (non-terminal symbol): là các tín hiệu ảo (**abstract**), không xuất hiện trong văn bản, nhưng là **trung gian** để xuất hiện các kí tự kết thúc (token).
 - Ngoài ra, CFG còn có định nghĩa về **tín hiệu bắt đầu (start symbol)** là kí hiệu đứng đầu CFG, là một **non-terminal symbol**.
- Rules:** CFG **dựa trên luật sinh** để minh họa các sequence of tokens. Luật sinh là một quy tắc sinh từ tín hiệu không kết thúc (non-terminal) trở thành một bộ tín hiệu mới, có thể chứa cả tín hiệu kết thúc lẫn không kết thúc.

- Gọi bộ tín hiệu kết thúc (các tokens) là T , bộ tín hiệu không kết thúc có kí hiệu là N . Giả sử bộ production rules của một ngôn ngữ có kí hiệu là P , trong P chứa các bộ luật sinh p khác nhau. Khi đó, p được biểu diễn bằng $X \rightarrow \alpha$, trong đó $X \in N$ (không kết thúc), còn $\alpha \in (T \mid N)^+$ (cả kết thúc và không kết thúc).
- **Components:** Như vậy, các thành phần của văn phạm phi ngữ cảnh bao gồm:
 - Bộ tín hiệu không kết thúc - non-terminal symbol N .
 - Bộ tín hiệu kết thúc - terminal symbol T .
 - Tín hiệu bắt đầu $S \in N$.
 - Bộ luật sinh P . Trong đó có các luật sinh $p \in P : X \rightarrow \alpha$.

Context-free Grammar: Examples

- Trong ngôn ngữ C, ta có thể sử dụng CFG để biểu thị một vòng lặp `while` như sau:

$$while_statement \longrightarrow WHILE (expression) block_statement$$

- Trong đó:
 - Các tín hiệu không kết thúc: `while_statement`, `expression` (biểu thức), `block_statement`
 - Các tín hiệu kết thúc: `while`, `(`, `)`
- Ta có thể định nghĩa tiếp các tín hiệu không kết thúc, ví dụ như `block_statement` :

$$block_statement \longrightarrow \{ statement \} \text{ or } \{ \}$$

- Trong đó:
 - Các tín hiệu không kết thúc: `block_statement`, `statement`
 - Các tín hiệu kết thúc: `{`, `}`
- Có thể hiểu: Ở trong một `block_statement` có thể là một chuỗi các trạng thái (`statement`) hoặc không có gì cả (rỗng).

Backus-Naur Form (BNF) Vs. Extend Backus-Naur Form (EBNF)

- **BNF là loại kí hiệu văn phạm phi ngữ thuần túy nhất**, được thực hiện bởi 2 tác giả của ngôn ngữ Algol-60.
- **BNF định nghĩa CFG rất chặt**, tức **không được sử dụng các toán tử trong RegEx** và các phân cấp cao cấp hơn không được sử dụng trong BNF.
- Muốn chuỗi lặp đi lặp lại trong BNF, ta **bắt buộc phải tạo ra các luật sinh đệ quy (recursive rules)**.
- **EBNF là một dạng extend của BNF**, cho phép sử dụng các toán tử RegEx vào nhằm mục đích đơn giản hóa cách định nghĩa ngữ pháp.
- **EBNF giúp cho CFG dễ đọc, dễ hiểu hơn, diễn đạt những quy tắc khó một cách dễ dàng hơn.**

- Về mặt nguyên tắc, **mọi CFG biểu diễn được bằng ENBF đều biểu diễn được bằng BNF, tuy nhiên EBNF sẽ ngắn gọn hơn.**
- **So sánh BNF và EBNF:**
 - Về các **luật lặp lại (repetition)**, BNF sử dụng luật sinh đệ quy (recursive rules), còn EBNF cho phép sử dụng RegEx operator (`*`, `+`, `?`...)
 - Về **quy tắc nhóm (grouping)**, BNF cần phải định nghĩa nhiều tín hiệu không kết thúc (non-terminal symbol) để tách các nhóm ra định nghĩa riêng, còn EBNF cho phép sử dụng các dấu ngoặc đơn `()` (parentheses) để ghi thẳng trong 1 công thức.
 - Về các **lựa chọn của toán tử or (alternatives)**, BNF cho phép sử dụng toán tử `|`, EBNF cũng như thế, và cho phép thêm grouping để nhóm các toán tử cần lựa chọn vào một group.
 - Về các **toán tử tùy chọn (trường hợp đặc biệt của alternative, bao gồm cả toán tử rỗng ϵ)**, ở BNF có thể sử dụng toán tử `|` kết hợp với ϵ , còn ENBF sử dụng cặp ngoặc vuông `[]`.

Parser: How it works

How a parser uses CFG

- Trong môn học, ta chỉ mô tả CFG, sau đó nhờ ANTLR để generate ra parse tree.
- Quá trình thực hiện từ tokens sang parse tree:
 - Bắt đầu với tín hiệu bắt đầu (start symbol)
 - Từ start symbol, ta áp dụng luật sinh (production rules) để mở rộng dần các tín hiệu không kết thúc (non-terminal symbol).
 - Kết nối các tokens từng bước một, cho tới khi input được hình thành đầy đủ.
 - Nếu không có dẫn xuất hợp lệ nào tồn tại, trả về thông báo `syntax error`.
 - Kết quả: một parse tree được hình thành từ sequence of tokens (khi input được chấp nhận) hoặc một thông báo `syntax error` (khi input vi phạm quy tắc ngữ pháp)

→ Quá trình này gọi là một **top-down parser**, tức sinh cây theo thứ tự từ trên xuống dưới.

Leftmost Derivation

- **Dẫn xuất trái nhất (leftmost derivation) là một chiến lược dẫn xuất trong CFG**, sử dụng các **production rules** để chuyển dần **sentential form (có cả terminal và non-terminal) thành chuỗi chỉ gồm terminal**.
- Phương thức này sinh ra cây parse tree bằng cách **luôn luôn mở rộng non-terminal ở bên trái nhất, sử dụng CFG (dạng BNF/EBNF) để replace non-terminal theo luật sinh**. Cứ như vậy, thực hiện theo quy tắc đệ quy cho đến khi toàn bộ cấu trúc được chấp nhận (toàn bộ output đều là terminal).
- Các lợi ích khi sử dụng Leftmost derivation:
 - Có tính quyết định: Bảo đảm một quá trình dẫn xuất rõ ràng, từng bước

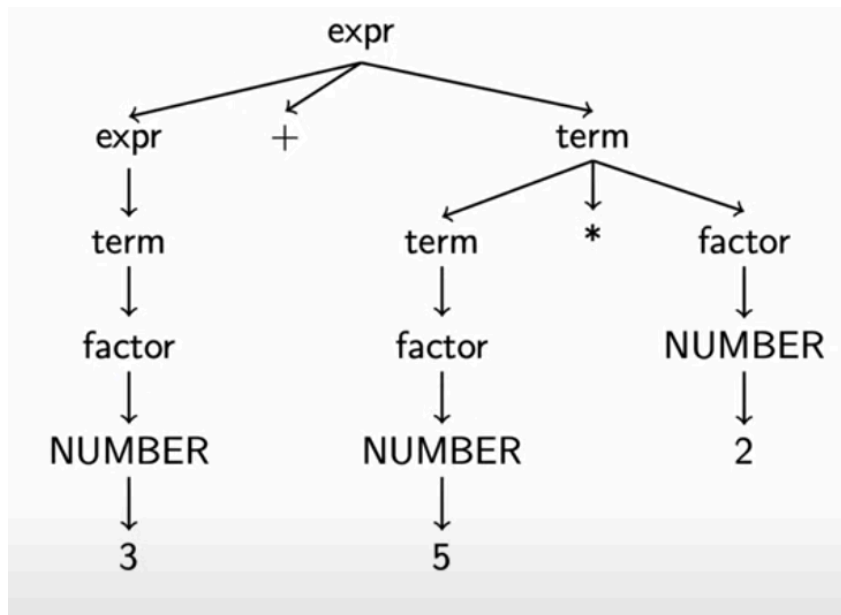
- Dễ dàng hiện thực theo top-down parser, tức là thống nhất parse tree theo một quy tắc thay vì chọn non-terminal ngẫu nhiên không theo quy luật.
- Đoán trước cây sẽ được hình thành như thế nào.

Leftmost Derivation: Examples

Giả sử ta có một bộ luật sinh cho các biểu thức, được viết theo dạng EBNF như sau:

$$\begin{aligned} \text{expr} &\longrightarrow \text{expr} + \text{term} \mid \text{term} \\ \text{term} &\longrightarrow \text{term} * \text{factor} \mid \text{factor} \\ \text{factor} &\longrightarrow (\text{expr}) \mid \text{NUMBER} \end{aligned}$$

- quy ước 3 rules trên lần lượt là rule 1, rule 2, rule 3. Mỗi rule có vế trái và vế phải.
- input giả định là $3 + 5 * 2$.
- **Sử dụng bộ production rules trên kết hợp với chiến lược leftmost derivation**, ta có các bước thực hiện sau đây:
 1. Bắt đầu với start symbol là *expr*.
 2. Phép tính trên là phép cộng của 2 phần tử → chọn vế đầu rule 1 của *expr*. *expr* $\longrightarrow$ *expr* + *term*
 3. Lấy bên trái nhất để extend → *expr*. Trong input ta thấy bên trái phép cộng là một scalar "3" → Chọn vế phải của rule 1. *expr* $\longrightarrow$ *term* + *term*
 4. Tiếp tục lấy bên trái nhất để extend → lấy "term" ở bên trái dấu cộng. Vẫn tham chiếu tới scalar "3" → lấy vế phải của rule 2. *expr* $\longrightarrow$ *factor* + *term*
 5. Lấy bên trái nhất để extend → *factor*. Tham chiếu tới scalar "3" → Lấy vế trái của Rule 3 tức "NUMBER" - là một token. Trong parse tree, ta dừng đào sâu nhánh này ở đây, chuyển ngược lên nhánh còn lại theo quy tắc đệ quy. *expr* $\longrightarrow$ 3 + *term*
 6. Lấy non-terminal bên trái nhất lúc này - "term". Tham chiếu tới input, ta thấy còn lại một phép nhân giữa 2 phần tử → Chọn vế bên trái của rule 2. *expr* $\longrightarrow$ 3 + *term* * *factor*.
 7. Tiếp tục lấy non-terminal bên trái nhất: "term". Tham chiếu tới input, ta thấy scalar "3" → lấy vế phải của rule 2. *expr* $\longrightarrow$ 3 + *factor* * *factor*.
 8. Tiếp tục lấy non-terminal bên trái nhất: *factor* bên trái dấu *. Tham chiếu tới input, ta thấy scalar "3" → lấy vế phải của rule 3. *expr* $\longrightarrow$ 3 + 5 * *factor*.
 9. Lấy non-terminal bên trái nhất, cũng là non-terminal cuối cùng: "factor" còn lại. Tham chiếu đến input, ta thấy scalar "2" → lấy vế bên phải của rule 3. *expr* $\longrightarrow$ 3 + 5 * 2.
Kết thúc.
- Áp dụng các rules trên và biểu diễn bằng tree, ta được cây cú pháp:



Parser Generation From ANTLR4

- Ta chỉ viết CFG, rồi nhờ ANTLR4 generated ra parse tree.
- Các quy tắc viết parser trong ANTLR4:
 - Viết rules bằng chữ thường (phân biệt với chữ in hoa trong lexer)
 - Trong quá trình sinh, vế phải (sau dấu mũi tên) có thể là parser rules hoặc lexer rules.
 - Dấu mũi tên (sinh) được biểu thị bằng dấu `:` trong ANTLR4.
 - Alternative được biểu thị bằng dấu `|` trong ANTLR4.
 - Sự lặp lại của các phần tử được biểu diễn như RegEx: `*`, `+`, `?`.
 - Nhóm các phần tử bằng dấu `()`.
 - Token đặc biệt cho starting rules: Phải kết thúc rules này bằng token `EOF`.

- Giả sử với expression rules, ta code như sau:

```
grammar Arithmetic;
```

```
// parser Rules
```

```
expr: expr '+' term
    | expr '-' term
    | term;
```

```
term: term '+' factor
    | term '-' factor
    | factor;
```

```
factor: '(' expr ')'
      | NUMBER;
```

```
// lexer
NUMBER: [0-9]+;
WS: [ \r\n]+ → skip;
```

Challenges: Ambiguous Grammar

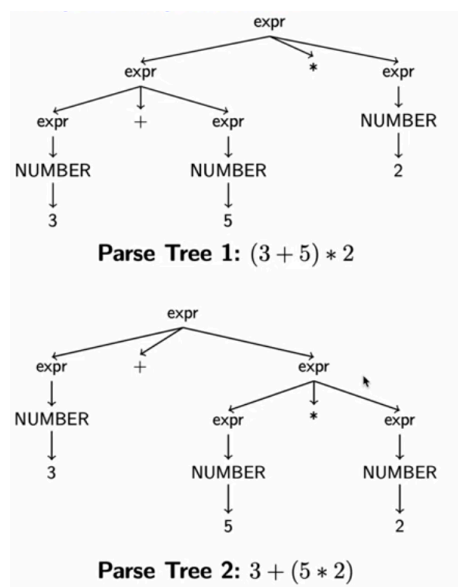
- **Vấn phạm nhập nhằng (Ambiguous grammar)** xảy ra khi tồn tại nhiều hơn 1 parse tree có thể sinh ra từ **production rules**. Điều này dẫn tới **Parser Conflict**, khiến cho parser “lúng túng” không biết sử dụng parse tree nào mới chính xác.
- **Vấn phạm nhập nhằng thường xảy ra khi các rules có tính đối xứng (symmetric)**, tiêu biểu là trường hợp:
 - Không quy định độ ưu tiên của phép nhân/chia so với phép cộng/trừ, tức là khai báo chung nhân chia và cộng trừ trong một rule mà không tách ra và ưu tiên một trong hai.

```
grammar AmbiguousExpr;

expr: expr ( '+' | '-' ) expr // AddSub
    | expr ( '*' | '/' ) expr // MulDiv
    | '(' expr ')'             // Parens
    | NUMBER;                 // Number

NUMBER: [0-9]+;
WS: [ \t\r\n]+ -> skip;
```

Ambiguous Grammar



two possible Parse Trees

Common Patterns in Writing Parser Rules

- Trong quá trình viết parser, cách tiếp cận đặc trưng là tuân theo thứ tự xuất hiện của chúng trong đặc tả ngôn ngữ.



- Trong suốt quá trình viết parser, có 5 mẫu đệ quy thường gặp nhất là (gọi x là một tín hiệu kết thúc hoặc không kết thúc bất kỳ):
 - Một danh sách không rỗng các x .
 - Một danh sách có thể rỗng các x .
 - Một danh sách không rỗng các x cách nhau bởi y (y có thể là dấu phẩy, chấm phẩy ...)
 - Một danh sách có thể rỗng các x cách nhau bởi y (y có thể là dấu phẩy, chấm phẩy ...)
 - Một biểu thức trung tố (infix expression)

Non-empty list

Ví dụ cụ thể (ngôn ngữ C):

- Một chương trình của ngôn ngữ C phải có ít nhất là một hàm (main).
- Có thể nói một hàm trong ngôn ngữ C là một tín hiệu không kết thúc (non-terminal symbol), và **một program của ngôn ngữ C là một danh sách không rỗng các hàm (function).**
- Như vậy, ta có thể viết các production rule đại diện cho program of C Language như sau:

```
// BNF
program: func_list EOF;
func_list: func func_list | func;    // BNF must call itself recursively

// EBNF
program: func_list EOF;
func_list: func+;                    // EBNF can use '+' operator for non-empty list
```

- Công thức chung cho non-empty list x :
 - BNF: `xlist: x xlist | x;`
 - EBNF: `xlist: x+;`

Null-able list

Ví dụ cụ thể (ngôn ngữ C):

- Trong ngôn ngữ C , **thân của hàm (function) là một list có thể rỗng của các trạng thái (statement - stmt)**. Lưu ý, stmt được định nghĩa bao gồm cả semicolon operator (phân biệt với expression), nên có thể coi các statement không bị ngăn cách bởi bất kì operator nào.
- Như vậy, ta có thể viết các production rules đại diện cho program C Language như sau:

```
// BNF
program: func '(' ')' func_body EOF;
func_body: '{' stmtlist '}';
stmtlist: stmt stmtlist | ;

// EBNF
program: func '(' ')' func_body EOF;
funcbody: '{' stmtlist '}';
stmtlist: stmt*;
```



```
// example
void f() { }           // stmtlist = ε
void f() { x = 1; }    // stmtlist = stmt
void f() { x = 1; y = 2; } // stmtlist = stmt stmt
```

- Công thức chung cho null-able list x:
 - BNF: `xlist: x xlist | ;`
 - EBNF: `xlist: x*;`

Non-empty list, seperated by operator

Ví dụ cụ thể (ngôn ngữ C):

- Một khai báo biến (variable declaration) trong C bắt đầu với kiểu dữ liệu (INT, FLOAT, ...), sau đó là danh sách các identifiers cách nhau bởi dấu phẩy. Như vậy, **một khai báo biến trong C tồn tại dưới dạng một danh sách không rỗng, cách nhau bởi dấu chấm phẩy (semicolon)**.
- Trong CFG, rule về variable declaration thường được biểu thị dưới dạng *vardecl*.
- Giả sử ta đang muốn viết production rules cho khai báo biến. Trong đó:
 - ID là token đại diện cho identifier.
 - FLOAT, INT là các tokens đại diện cho data type
 - SEMI là token cho semicolon operator
 - COMMA là token của comma operator
- Khi đó, ta có thể viết mã ANTLR4 như sau:

```
// BNF
vardecl: (INT | FLOAT) idlist SEMI;
idlist: COMMA ID idlist | ID;

// ENBF
vardecl: (INT | FLOAT) idlist SEMI;
idlist: ID (COMMA ID)*;

// example
int a;
float x, y, z;
int i, j, k, l;
float score;
```

- Công thức chung cho null-able list x, seperated by y:

- BNF: `xlist: x y xlist | x;`
- EBNF: `xlist: x (y x)*;`

Null-able list, seperated by operator

Ví dụ cụ thể:

- Một khai báo tham số (parameter declaration) bắt đầu với một dấu ngoặc đơn trái `(`, sau đó là một list có thể rỗng các tham số đầu vào ngăn cách nhau bởi dấu chấm phẩy, cuối cùng là dấu ngoặc đơn phải `)`.
- Trong CFG, parameter declaration thường được viết production rule dưới dạng non-terminal là *vardecl*. Một biến thì được kí hiệu là *param*.
- Mã ANTLR4:

```
// BNF - 1
paramdecl: LB paramlist RB;
paramlist: paramprime | ;
paramprime: param SM paramprime | param;

// BNF - 2
paramdecl: LB paramlist RB;
paramlist: param smparam | ;
smparam: SEMI param smparam | ;

// EBNF - 1
paramdecl: LB paramlist RB;
paramlist: param (SEMI param)* | ;

// EBNF - 2
```

```
paramdecl: LB paramlist RB;  
paramlist: (param (SEMI param)*)?;
```

```
// result  
()  
(int x)  
(int x, float y)  
(int x, float y, char c)
```

- công thức chung cho null-able list x, separated by y:

- BNF:

```
xlist: xprime | ;  
xprime: x y xprime | x;
```

- EBNF:

```
xlist: x (y x)* | ;  
  
// or  
xlist: (x (y x)*)?;
```

An infix expression

- **Một biểu thức thường tính theo quy tắc nhân chia trước - cộng trừ sau. Thứ tự tính toán rất quan trọng.**
- Biểu thức trung tố đơn giản cho con người, nhưng không đơn giản cho máy tính. Biểu diễn ở dạng tiền tố và hậu tố có thể sử dụng stack để tính toán theo thứ tự tính toán, nhưng biểu thức trung tố thì không.
- **Khi tính toán biểu thức trung tố, cần xác định rõ thứ tự ưu tiên và tính kết hợp của các phép toán. Những quy tắc này được phản ánh trong parse tree khi phân tích cú pháp (syntax analysis).**
- Cho một mã ANTLR của production rules for infix expression như sau:

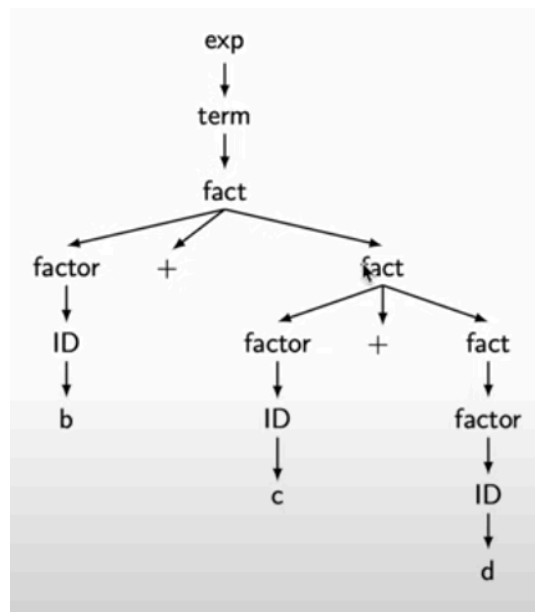
```
// parser  
grammar exp;  
exp: term ASSIGN exp | term;  
term: term EXPONENT fact | fact;  
fact: factor RELOP fact | factor ADDOP fact | factor;  
factor: ID;  
  
// lexer
```

```

ASSIGN: '=';
EXPONENT: '^';
ADDOP: '+';
RELOP: '>';
ID: [a-zA-Z_][a-zA-Z0-9_]*;
WS: [ \n\r\t]+ → skip;

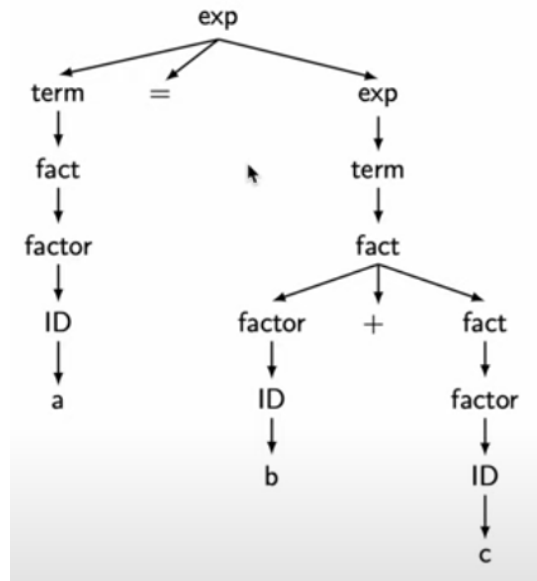
```

- Xét ví dụ về tính kết hợp. Với input là `b + c + d` , ta sinh ra được cây cú pháp sau:



Parse tree generated by input a + b + c

- Dựa vào cây, ta có nhận xét về **tính kết hợp**:
 - Tính biểu thức `c + d` trước tiên. Tức là parse tree kết hợp phải.
 - Non-terminal `fact` đang được đệ quy bên phải dấu `+` . Vậy, parse tree này đệ quy phải.
 - Như vậy, ta có kết luận: **Đệ quy phải thì có tính kết hợp phải.**
- Xét ví dụ về độ ưu tiên. Với input là `a = b + c` , ta sinh được cây cú pháp sau:



Parse Tree generated by input $a = b + c$

- Dựa vào cây, ta có nhận xét về độ ưu tiên:
 - token ASSIGN cao hơn toán tử ADD trong cây, nên độ ưu tiên thấp hơn
 - Kết luận: **Operator càng sâu so với tín hiệu bắt đầu, độ ưu tiên càng lớn. Operator nằm càng gần tín hiệu bắt đầu, độ ưu tiên càng thấp.**



- **Tổng kết:**

- **Về Độ ưu tiên của toán tử (operator precedence):**
 - Toán tử nằm càng gần tín hiệu bắt đầu, độ ưu tiên càng thấp.
 - Toán tử nằm càng xa (độ sâu càng lớn) so với tín hiệu bắt đầu, độ ưu tiên càng cao.
- **Về Tính kết hợp của toán tử (operator associativity):**
 - Độ quy trái thì kết hợp trái
 - Độ quy phải thì kết hợp phải
 - Không độ quy thì không có tính kết hợp.

Trắc nghiệm

1. Trong quá trình dịch, bước phân tích cú pháp là bước:

- a. Đầu tiên
- b. Trước bước phân tích từ vựng
- c. **Ngay sau bước phân tích từ vựng**
- d. Sau bước sinh mã trung gian

2. Vai trò của bộ phân tích cú pháp gồm:

- a. Tách chuỗi nhập thành các nhóm
- b. **Xác định trình tự các tokens có đúng với các luật văn phạm không**
- c. **Tạo ra cây phân tích cú pháp nếu chuỗi tokens đúng văn phạm**
- d. Kiểm tra liệu một danh hiệu có được khai báo trước khi dùng không

3. Cho các luật sinh của văn phạm như sau:

$M \rightarrow a X b$

$X \rightarrow c Y \mid d$

$Y \rightarrow m Y \mid \epsilon$

Kí hiệu bắt đầu của văn phạm trên là? $\rightarrow M$

4. Cho các luật sinh của văn phạm như sau:

$M \rightarrow a X b$

$X \rightarrow c Y \mid d$

$Y \rightarrow m Y \mid \epsilon$

Dùng các luật sinh này, theo thứ tự alphabet và cách nhau bằng dấu phẩy, tập các kí hiệu không kết thúc của văn phạm trên là? $\rightarrow \{M, X, Y\}$

5. Cho các luật sinh của văn phạm như sau:

$M \rightarrow a X b$

$X \rightarrow c Y \mid d$

$Y \rightarrow m Y \mid \epsilon$

Hãy cho biết tập các ký hiệu kết thúc của văn phạm trên?

- a. $\{a,b,c,d,m,\epsilon\}$
- b. $\{M,a,b,X,c,d,Y,m\}$
- c. $\{a,b,c,d\}$

d. $\{a,b,c,d,m\}$

6. Cho các luật sinh của văn phạm như sau:

$M \rightarrow a X b$

$X \rightarrow c Y \mid d$

$Y \rightarrow m Y \mid \epsilon$

Hãy cho biết số các luật sinh trong văn phạm ở câu 5?

a. 3

b. 4

c. 5

d. 6

7. Cho các luật sinh của một văn phạm như sau:

$M \rightarrow M X \mid Y$

$X \rightarrow c X \mid d$

$Y \rightarrow a$

Hãy chọn các quá trình dẫn xuất đúng để sinh ra chuỗi sau: $acdccd$

a. $M \Rightarrow MX \Rightarrow MXX \Rightarrow YXX \Rightarrow aXX \Rightarrow acXX \Rightarrow acdX \Rightarrow acdcX \Rightarrow acdccX \Rightarrow acdccd$

b. $M \Rightarrow MX \Rightarrow McX \Rightarrow Mcd \Rightarrow McdX \Rightarrow McdcX \Rightarrow McdccX \Rightarrow Mcdccd \Rightarrow Ycdccd \Rightarrow acdccd$

c. $M \Rightarrow MX \Rightarrow McX \Rightarrow MccX \Rightarrow Mccd \Rightarrow MXccd \Rightarrow McXccd \Rightarrow Mcdccd \Rightarrow Ycdccd \Rightarrow acdccd$

d. $M \Rightarrow MX \Rightarrow MXX \Rightarrow McXcX \Rightarrow Mcdccd \Rightarrow Ycdccd \Rightarrow acdccd$

8. Cho các luật sinh của văn phạm G như sau:

$M \rightarrow M X \mid Y$

$X \rightarrow c X \mid d$

$Y \rightarrow a$

Hỏi ngôn ngữ $L(G)$ được biểu diễn bởi văn phạm trên là gì?

a. $L(G)$ cũng được mô tả bởi biểu thức chính qui $a(c+d)^+$

b. (G) cũng được mô tả bởi biểu thức chính qui $a(c*d)^+$

c. $L(G)$ cũng được mô tả bởi biểu thức chính qui $a(c+d)^*$

d. $L(G)$ cũng được mô tả bởi biểu thức chính qui $a(c*d)^*$

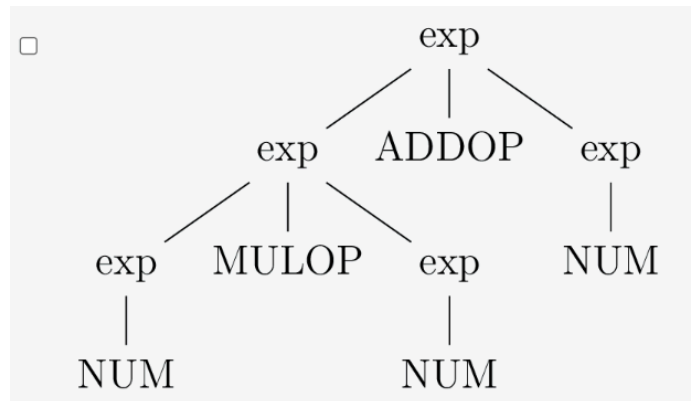
9. Sử dụng văn phạm của biểu thức được cho trong bài giảng.

$\langle \text{exp} \rangle \rightarrow \langle \text{exp} \rangle \text{ ADDOP } \langle \text{exp} \rangle$
 $\mid \langle \text{exp} \rangle \text{ MULOP } \langle \text{exp} \rangle$

| LB <exp> RB
| INTLIT

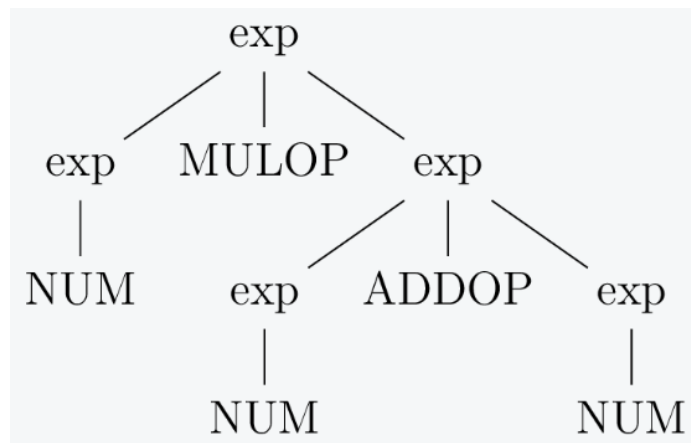
Với chuỗi nhập $12 * 4 + 3$, sau bước phân tích từ vựng, chuỗi tokens được cung cấp cho bộ phân tích từ vựng là NUM MULOP NUM ADDOP NUM. Hãy cho biết các cây phân tích cú pháp của chuỗi tokens này hợp lệ đối với văn phạm được cho?

a. Chọn



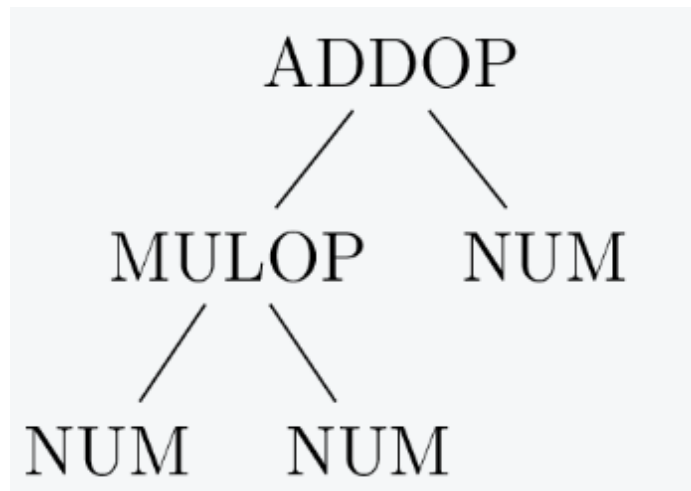
(Chọn ADDOP đi trước, sinh ra NUM MULOP NUM ADDOP NUM, đúng với lexer)

b. Chọn



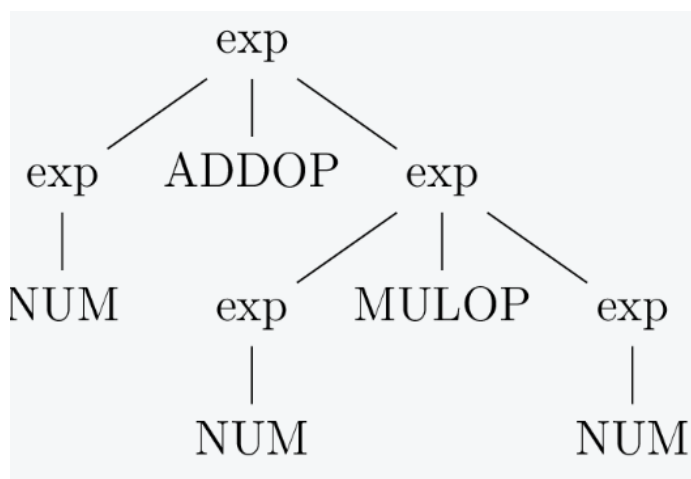
(Chọn MULOP đi trước, vẫn sinh ra NUM MULOP NUM ADDOP NUM)

c. Chọn



(Sai vì đây là AST Tree, không phải parse tree)

d. Chọn



(sai vì câu này ra token NUM ADDOP NUM MULOP NUM)

10. **Chọn True nếu văn phạm bị nhập nhằng và False nếu văn phạm không bị nhập nhằng**

$S \rightarrow SA \mid A$
 $A \rightarrow Aa \mid b$

- a. True
- b. **False**

$S \rightarrow SA \mid A$
 $A \rightarrow Aa \mid a$

- a. **True**

b. False

$S \rightarrow c S S \mid b$

a. True

b. **False**

$S \rightarrow S c S \mid b$

a. **True**

b. False

11. Cho phép toán - có tính kết hợp trái và phép toán / có tính kết hợp phải và phép toán / (chia nguyên) có độ ưu tiên cao hơn phép toán -. Hãy cho biết kết quả của biểu thức: $100 / 20 / 5 - 32 / 8 / 2 - 9 / 3 / 3$

a. -2

b. -1

c. **8**

d. Một giá trị khác

- Ưu tiên phép chia trước phép trừ → thực hiện chia trước.
- 2 phép chia liên tiếp thì thực hiện phép chia bên phải trước (vì kết hợp phải)
- 2 phép trừ liên tiếp thì thực hiện bên trái trước (vì kết hợp trái)
- Từ đó ta có:
 - $(100 / (20 / 5) - 32 / (8 / 2)) - 9 / (3 / 3)$
 - $(100 / 4 - 32 / 4) - 9 / 1$
 - $(25 - 8) - 9$
 - $17 - 9 = 8.$

12. Cho biểu thức có các phép toán AND, OR và XOR trong đó phép toán and có tính kết hợp phải và ưu tiên thấp nhất, phép toán OR có tính kết hợp trái và có độ ưu tiên trung bình, còn phép toán XOR không có tính kết hợp và có độ ưu tiên cao nhất. Hãy viết các luật sinh sao cho đảm bảo được độ ưu tiên và tính kết hợp của các phép toán (Đề ghi về phải trùng khớp, viết các ký hiệu văn phạm cách nhau 1 khoảng trắng, viết hoa các ký hiệu kết thúc, viết chữ thường cho các ký hiệu không kết thúc).

Cho exp0 là ký hiệu bắt đầu.

1. $\text{exp0} \rightarrow __ \mid \text{exp1}$

2. $\text{exp1} \rightarrow __ \mid \text{exp2}$

3. $\text{exp2} \rightarrow __ \mid \text{exp3}$

Điền vào chỗ trống? →

1. **exp1 AND exp0**

2. **exp1 OR exp2**

3. **exp3 XOR exp3**

• Giải thích:

◦ **Sắp xếp theo độ ưu tiên:**

- **AND có độ ưu tiên thấp nhất nên sẽ ở gần kí hiệu bắt đầu nhất → luật 1**
- **AND có độ ưu tiên trung bình nên sẽ ở luật 2**
- **XOR có độ ưu tiên cao nhất nên sẽ xa kí hiệu bắt đầu nhất → luật 3**

◦ Tiếp tục sắp xếp vị trí:

- Luật 1 có 2 non-terminal là (exp1, exp0). Mà AND kết hợp phải nên đệ quy phải → exp0 (phần tử đệ quy) nằm bên phải của AND → **exp1 AND exp0.**
- Luật 2 có 2 non-terminal là (exp1, exp2). Mà OR kết hợp trái nên đệ quy trái → exp1 (phần tử đệ quy ở rule 2) nằm bên trái của OR → **exp1 OR exp2.**
- Luật 3 có 2 non-terminal là (exp2, exp3). Tuy nhiên, XOR không có tính kết hợp nên không có tính đệ quy, nghĩa là phần tử đệ quy (exp2) không xuất hiện ở vế phải → XOR giữa 2 phần tử exp3 với nhau. **exp3 XOR exp3.**

1. "cách nhau bởi dấu phẩy" + "ít nhất một ID"

Hãy sử dụng dạng **BNF** để viết các **vế phải** của các luật sinh mô tả một danh sách các danh hiệu, trong đó các danh hiệu được cách nhau bằng một dấu phẩy. Danh sách phải có ít nhất 1 danh hiệu. Ký hiệu văn phạm (không kết thúc) đại diện cho danh sách các danh hiệu là **idlist**, ký hiệu văn phạm đại diện cho danh hiệu là **ID**, ký hiệu văn phạm đại diện cho dấu phẩy là **COMMA**. Cho vế trái các luật sinh là **idlist** ?

Qui ước viết:

- Các ký hiệu văn phạm trong mỗi vế phải cách nhau đúng bằng 1 khoảng trắng
- Nếu có nhiều vế phải thì phải viết các vế phải cách nhau bằng 1 khoảng trắng, 1 dấu | và sau đó là 1 khoảng trắng
- Vế phải có nhiều ký hiệu hơn được viết trước vế phải có ít ký hiệu
- Nếu vế phải là rỗng thì ghi chữ **empty**
- Phải viết đúng các ký hiệu văn phạm được cho.

Trả lời:

idlist có "ít nhất một ID" → " ... idlist | ID"

Các ID cách nhau bởi dấu phẩy → ID COMMA

Gộp lại → ID COMMA idlist | ID

2. "có thể rỗng" + "các exp cách nhau bằng dấu phẩy"

Một danh sách các biểu thức bao gồm các biểu thức cách nhau bằng dấu phẩy và danh sách các biểu thức có thể rỗng. Ký hiệu văn phạm (không kết thúc) đại diện cho danh sách các biểu thức là `explist`, ký hiệu văn phạm (không kết thúc) đại diện cho một biểu thức là `exp`, ký hiệu văn phạm đại diện cho dấu phẩy là `COMMA`.

Hãy điền vào vế phải còn trống dưới đây ở dạng BNF để hoàn tất các luật văn phạm mô tả một danh sách các biểu thức.

`explist` -> `exp exprime | ε`

`exprime` ->

Quý ước viết:

- Các ký hiệu văn phạm trong mỗi vế phải cách nhau đúng bằng 1 khoảng trắng
- Nếu có nhiều vế phải thì phải viết các vế phải cách nhau bằng 1 khoảng trắng, 1 dấu `|` và sau đó là 1 khoảng trắng
- Vế phải có nhiều ký hiệu hơn được viết trước vế phải có ít ký hiệu
- Nếu vế phải là rỗng thì ghi chữ **empty**
- Phải viết đúng các ký hiệu văn phạm được cho (`explist`, `exp`, `exprime`, `COMMA`)

Trả lời:

I

Cách viết của đề là `explist` → `exp exprime | empty`, như vậy đã có trường hợp `empty` (rỗng) và phía trước cùng đã có một biểu thức.

Cần thêm: dấu phẩy + biểu thức (`COMMA exp`), đệ quy lại cái này (`exprime`). Hoặc là không thêm gì cả (rỗng). Như vậy sẽ đủ 3 trường hợp: rỗng, 1 phần tử, nhiều phần tử.

→ Thêm **COMMA exp exprime | empty**.

Ngoài ra, nếu đề hỏi `explist` → `exprime | empty` và viết đệ quy của `exprime`, đáp án sẽ là:
exp COMMA exprime | exp.

3. Danh sách các phát biểu "Các phát biểu đều kết thúc bằng dấu ; " "có thể rỗng"

Hãy sử dụng dạng **BNF** để viết các vế phải của các luật sinh mô tả một danh sách các phát biểu, trong đó các phát biểu đều được kết thúc bằng dấu `;`. Danh sách có thể không có phát biểu nào. Ký hiệu văn phạm (không kết thúc) đại diện cho danh sách các phát biểu là `stmtlist`, ký hiệu văn phạm đại diện cho một phát biểu là `stmt`, ký hiệu văn phạm đại diện cho dấu chấm phẩy là `SEMI`. Cho vế trái các luật sinh là `stmtlist` ?

Quý ước viết:

- Các ký hiệu văn phạm trong mỗi vế phải cách nhau đúng bằng 1 khoảng trắng
- Nếu có nhiều vế phải thì phải viết các vế phải cách nhau bằng 1 khoảng trắng, 1 dấu `|` và sau đó là 1 khoảng trắng
- Vế phải có nhiều ký hiệu hơn được viết trước vế phải có ít ký hiệu
- Nếu vế phải là rỗng thì ghi chữ **empty**
- Phải viết đúng các ký hiệu văn phạm được cho.

Answer:

I

- `stmtlist` "Có thể rỗng" → kết thúc bằng `" ..... stmtlist | empty"`
- Các phát biểu (`stmt`) đều kết thúc bằng dấu `;` → `stmt SEMI`
- Gộp lại: **stmt SEMI stmtlist | empty**

4. Dẫn xuất trái nhất.

Cho các luật sinh của văn phạm G như sau:

$S \rightarrow BA$

$A \rightarrow aA \mid a$

$B \rightarrow b \mid c$

Hãy chọn **dẫn xuất trái nhất** để G sinh ra được chuỗi **caaaa** ?

Chọn một:

- ☐ a. $S \Rightarrow BA \Rightarrow BaA \Rightarrow BaaA \Rightarrow caaaA \Rightarrow caaaA \Rightarrow caaaa$
- ☐ b. $S \Rightarrow BA \Rightarrow cA \Rightarrow caaaa$
- ☐ c. $S \Rightarrow BA \Rightarrow cA \Rightarrow caA \Rightarrow caaA \Rightarrow caaaA \Rightarrow caaaa$
- ☐ d. $S \Rightarrow BA \Rightarrow BaA \Rightarrow BaaA \Rightarrow BaaaA \Rightarrow Baaaa \Rightarrow caaaa$

- Hễ có nhiều in hoa mà in hoa trái nhất k được dẫn xuất thì loại.
 - câu a loại ở phần tử 3
 - câu b loại ở phần tử 4 (không có luật nào từ A ra aaaa)
 - **câu C được. CHỌN C.**
 - câu d loại phần tử 3

5. Chuỗi thuộc luật sinh

- **"aaab"** nhận. $S \rightarrow AB \rightarrow aAB \rightarrow aaAB \rightarrow aaaB \rightarrow aaab$
- **"ab"** nhận. $S \rightarrow AB \rightarrow aB \rightarrow ab$.
- **"a"** loại. Luật yêu cầu phải có b hoặc c trong văn phạm.
- d loại. kí tự rỗng là không có kí tự chứ không phải ghi vào như vậy.

6. Câu đố thị

Cho văn phạm phi ngữ cảnh G với tập ký hiệu kết thúc là {ASSIGN, EXPONENT, ADDOP, RELOP, LB, RB}, tập ký hiệu không kết thúc là {exp, term, fact}, ký hiệu bắt đầu là exp, và tập luật sinh là:

exp $\rightarrow$ term ASSIGN exp | term

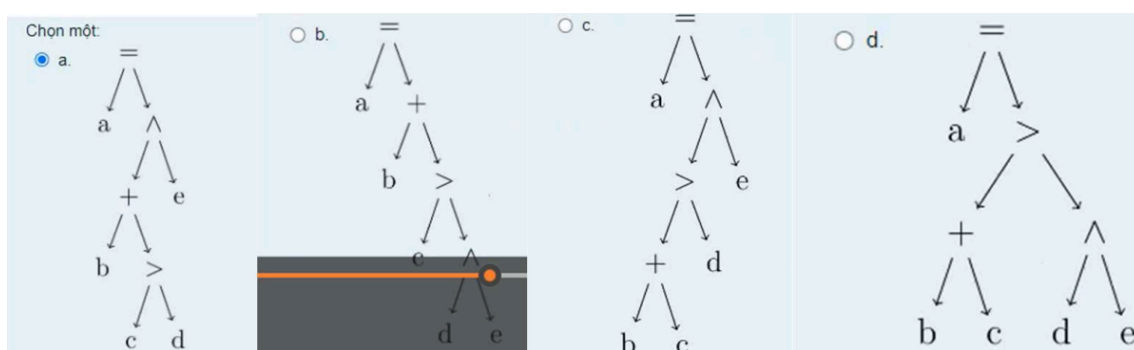
term $\rightarrow$ term EXPONENT fact | fact

fact $\rightarrow$ factor RELOP fact | factor ADDOP fact | factor

factor $\rightarrow$ LB exp RB | ID

Cho ID là token của các danh hiệu, ASSIGN của '=', EXPONENT của '^', ADDOP của '+' hay '-', RELOP của '>', LB của '(' và RB của ')'.
 Hãy xác định độ ưu tiên và tính kết hợp của các phép toán để từ đó xác định cây cú pháp trừu tượng (abstract syntax tree) của chuỗi nhập sau: a = b + c > d ^ e ?

Đọc kĩ đề. **Nếu đề vẫn như vậy thì đáp án là đồ thị có dấu > ở dưới cùng, phân chia thành c và d.** (câu a).



7. Lỗi từ cú pháp là?

Hãy chọn những lỗi được sinh ra bởi bước phân tích cú pháp?

Chọn một hoặc nhiều hơn:

- ☐ a. Dòng 3 cột 1: Kiểu của biểu thức không phù hợp
- ☐ b. Dòng 3 cột 1: Token HASH không được mong chờ tại đây
- ☐ c. Dòng 3 cột 1: Ký tự không được phép \$
- ☐ d. Dòng 3 cột 1: Biến a chưa được khai báo

- **Lỗi sinh ra từ phân tích cú pháp:**
 - **Token HASH không được mong chờ**
- Các lỗi kia:

- Kiểu của biểu thức không phù hợp → semantic analysis
- Ký tự không được phép \$ → lexical analysis
- Biến a chưa được khai báo → semantic analysis

8. Văn phạm nhập nhằng

Các văn phạm nhập nhằng:

$A \rightarrow AcA \mid b$ (đối xứng, ví dụ như bcbcb)

$S \rightarrow SS \mid a \mid b$

Còn lại không nhập nhằng.

9. Chọn cây phân tích cú pháp đúng.

Cho các luật sinh của văn phạm G như sau:

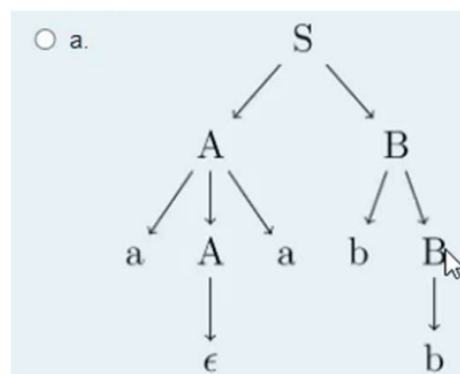
$S \rightarrow AB$

$A \rightarrow aAa \mid \epsilon$

$B \rightarrow bB \mid b$

Hãy chọn cây phân tích cú pháp cho chuỗi nhập **aabb** ?

Cây đúng có dạng:



10. Vai trò của bộ phân tích cú pháp (parser)

Hãy chọn các vai trò của bộ phân tích cú pháp?

Chọn một hoặc nhiều hơn:

- ☐ a. Xác định trật tự của chuỗi tokens có phù hợp với qui định của ngôn ngữ không
- ☐ b. Ghi nhận vị trí (hàng, cột) của các tokens
- ☐ c. Xây dựng cây phân tích cú pháp cho chuỗi tokens
- ☐ d. Trả về một chuỗi tokens ứng với chuỗi nhập

Vai trò đúng:

Xác định trật tự của chuỗi tokens

Xây dựng cây phân tích cú pháp.

Còn lại:

Ghi nhận hàng, cột → lexer

Trả về chuỗi tokens tương ứng với chuỗi nhập → lexer